# GREEDY MATCHING IN BIPARTITE RANDOM GRAPHS

NICK ARNOSTI, COLUMBIA BUSINESS SCHOOL

THIS VERSION: JUNE 2020

Abstract. This paper studies the performance of greedy matching algorithms on bipartite graphs $G = (\mathcal{J}, \mathcal{D}, \mathcal{E})$. We focus primarily on three classical algorithms: RANDOM-EDGE, which sequentially selects random edges from $\mathcal{E}$; RANDOM-VERTEX, which sequentially matches random vertices in $\mathcal{J}$ to random neighbors, and RANKING, which sequentially matches random vertices in $\mathcal{J}$ to the highest-priority remaining neighbor. Prior work has focused on identifying the worst-case approximation ratio for each algorithm. This guarantee is highest for RANKING, and lowest for RANDOM-EDGE. Our work instead studies the *average* performance of these algorithms when the edge set $\mathcal{E}$ is random.

Our first result establishes that on average, RANDOM-EDGE produces a smaller matching than RANDOM-VERTEX. In contrast to previous asymptotic results, this holds for finite graphs of arbitrary size, and with arbitrary degree distributions for vertices in $\mathcal{J}$. Furthermore, it applies in settings where vertices in $\mathcal{D}$ have arbitrary capacities, and (with minor modifications) to settings where vertices in $\mathcal{J}$ have different sizes and vertices in $\mathcal{D}$ have different expected degree.

Our second result is that when each vertex in $\mathcal{D}$ can be assigned to only one vertex in $\mathcal{J}$, any two vertex-iterative algorithms that consider the vertices of $\mathcal{J}$ in the same order lead to the same probability of matching for each vertex in $\mathcal{J}$. In particular, this implies that RANKING has the same average performance as RANDOM-VERTEX, despite offering a better worst-case guarantee.

## 1. Introduction

This paper studies the performance of greedy algorithms for many-to-one bipartite matching. While bipartite matching has many applications, we adopt the terminology of scheduling "jobs" on different "days." Although maximum matchings can be found in polynomial time, there has been considerable interest in understanding the performance of simple greedy algorithms. Performance is traditionally measured by the worst case ratio between the size of the matching produced by the algorithm and the size of a maximum matching. No deterministic greedy algorithm can provide a guarantee above 1/2, so attention has focused on randomized greedy algorithms.

One natural algorithm considers edges in a random order. We call this RANDOM-EDGE; it is referred to as "simple case algorithm" by Tinhofer (1984), and "greedy" by Dyer and Frieze (1991). Another algorithm schedules jobs sequentially, selecting among feasible days uniformly at random. We call this RANDOM-VERTEX; it is referred to as "modified random greedy" by Aronson et al. (1995) and "random" by Karp et al. (1990), and most subsequent work adopts one of these names. A third alternative sequentially schedules jobs using a universal ordering over days. This approach was introduced by Karp et al. (1990) under the name RANKING, which we also adopt.[1]

Dyer and Frieze (1991) show that the worst-case guarantee of RANDOM-EDGE is only 1/2. A series of papers (discussed in more detail below) have established increasingly tight bounds for the guarantees offered by RANDOM-VERTEX and RANKING; the tightest bounds of which we are

[1]Although we adopt the name RANKING, there is an important difference between our algorithm and that studied by Karp et al. (1990): whereas they assume that the order in which jobs are considered is adversarial, most subsequent work (including our own) assumes that this order is random. Similarly, the "random" algorithm of Karp et al. (1990) assumes an adversarial order of jobs, whereas "modified random greedy" randomizes this order. As we discuss in more detail below, randomizing arrival order allows for strictly better guarantees than those proved by Karp et al. (1990).

| | Worst Case Ratio ($C_d = 1$) |
|---|---|
| RANDOM-EDGE | 0.5 |
| RANDOM-VERTEX | [0.639, 0.646] |
| RANKING | [0.696, 0.724] |

TABLE 1. Worst-case approximation ratios for bipartite matching, from prior work. Although RANKING offers the best guarantee, on some graphs it is outperformed by RANDOM-VERTEX and RANDOM-EDGE (see Figure 1). We study the average performance of these algorithms, and find that when $C_d = 1$, RANKING and RANDOM-VERTEX have identical performance, and both outperform RANDOM-EDGE.

aware are presented in Table 1. These bounds establish that of these algorithms, RANKING offers the best guarantee, and RANDOM-EDGE the worst. Largely based on these worst-case bounds, graduate students who study online matching learn that RANKING is a better algorithm for online matching than RANDOM-VERTEX.[2] Meanwhile, the following heuristic argument "explains" the poor guarantee of RANDOM-EDGE: it disproportionately schedules high-degree jobs up front, even though low-degree jobs intuitively face the greatest risk of going unassigned.

This reasoning and the bounds in Table 1 notwithstanding, there are in fact graphs where RANDOM-VERTEX outperforms RANKING, and both are outperformed by RANDOM-EDGE (see Figure 1). How common are these cases, and what is true on a "typical" graph? Does worst-case analysis paint a representative picture of these algorithms' performance?

We tackle these questions by comparing these algorithms' *average* performance across all graphs where jobs have a specified degree sequence. Our first result is that the aforementioned intuition for the poor performance of RANDOM-EDGE can be made rigorous: for any specified degree sequence, the average performance of RANDOM-VERTEX exceeds that of RANDOM-EDGE. Our second result establishes that the apparent gap between RANDOM-VERTEX and RANKING vanishes when looking at average performance: for one-to-one matching, the two algorithms are equivalent.

In Section 1.1 we introduce our notation and formally define the algorithms RANDOM-EDGE, RANDOM-VERTEX, and RANKING. Section 1.2 describes what is known about the worst-case performance of these algorithms. Section 1.3 states our results, and compares these results to existing average-case analyses. We prove our main results in Sections 2 and 3.

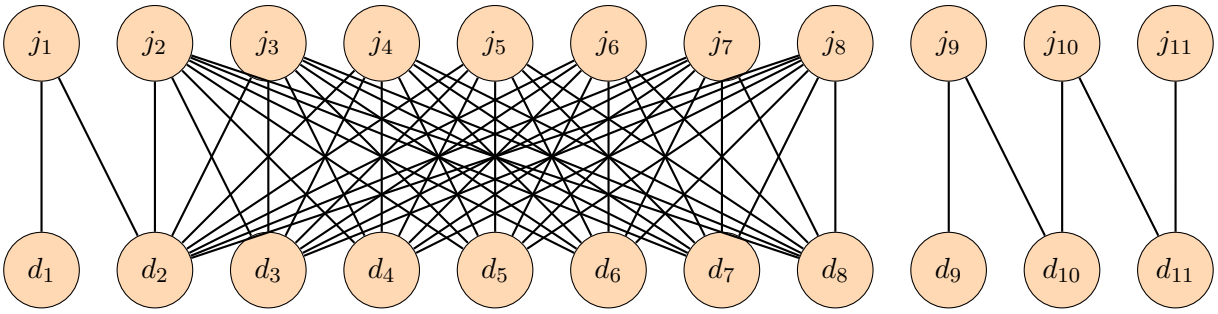
### 1.1. Preliminaries: Notation and Definitions.

We adopt the terminology of assigning a set of jobs $\mathcal{J}$ to a set of days $\mathcal{D}$. Each job has associated *scheduling* constraints, which are captured by the edge set $\mathcal{E} \subseteq \mathcal{J} \times \mathcal{D}$: the edge $(j, d) \in \mathcal{E}$ if and only if job $j$ can be scheduled for day $d$. Each day $d$ has an associated *capacity* constraint $C_d \in \mathbb{N}$, indicating the maximum number of jobs that can be scheduled on that day. We adopt the following definitions.

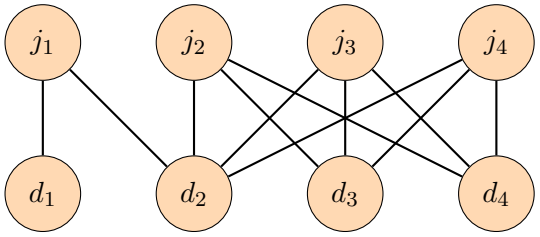
**Definition 1.** Fix $G = (\mathcal{J}, \mathcal{D}, \mathcal{E})$, and $\mathbf{C} \in \mathbb{N}^{\mathcal{D}}$.

- A **matching** is a set of edges $\mathcal{M} \subseteq \mathcal{J} \times \mathcal{D}$.
- A matching $\mathcal{M}$ is **feasible** (with respect to $\mathcal{E}$ and $\mathbf{C}$) if $\mathcal{M} \subseteq \mathcal{E}$, and in the graph $(\mathcal{J}, \mathcal{D}, \mathcal{M})$, each $j \in \mathcal{J}$ has at most one neighbor and each $d \in \mathcal{D}$ has at most $C_d$ neighbors.
- A matching $\mathcal{M}$ is **maximal** (with respect to $\mathcal{E}$ and $\mathbf{C}$) if it is feasible, and for all $e \in \mathcal{E} \backslash \mathcal{M}$, $\mathcal{M} \cup \{e\}$ is not feasible.
- A matching $\mathcal{M}$ is **maximum** (with respect to $\mathcal{E}$ and $\mathbf{C}$) if it is feasible, and there is no larger feasible matching.

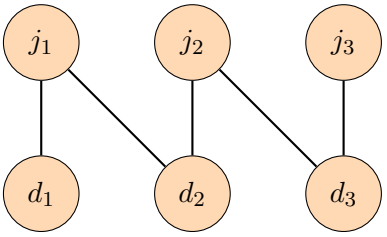
[2]For example, Fall 2019 lecture notes from UT Austin discuss RANDOM-VERTEX in a section titled "A (Bad) Randomized Algorithm", and introduce RANKING under the heading "The (Good) Randomized Algorithm." https://www.cs.utexas.edu/~ecprice/courses/randomized/notes/lec14.pdf




(A) Expected unassigned jobs is $< 0.704$ under RANDOM-EDGE, $17/24 \approx 0.708$ under RANDOM-VERTEX, and $17/24 + 1/48 \approx 0.729$ under RANKING.




(B) Expected unassigned jobs:
$5/24 \approx 0.21$ under RANKING,
$8/33 \approx 0.24$ under RANDOM-EDGE, and
$1/4 = 0.25$ under RANDOM-VERTEX.




(C) Expected unassigned jobs:
$11/24 \approx 0.46$ under RANDOM-VERTEX,
$8/15 \approx 0.53$ under RANDOM-EDGE, and
$13/24 \approx 0.54$ under RANKING.

FIGURE 1. Comparisons between RANKING, RANDOM-VERTEX, RANDOM-EDGE. Although worst-case analysis suggests that RANKING performs best and RANDOM-EDGE worst (see Table 1), this ordering is reversed on the graph in 1a. For any pair of these algorithms, one performs better on the graph in 1b, and the other performs better on the graph in 1c.

This paper considers greedy algorithms which are parameterized by a complete order $\succ^E$ on $\mathcal{J} \times \mathcal{D}$. These algorithms start with $\mathcal{M} = \emptyset$, and in the order given by $\succ^E$, sequentially add edges to $\mathcal{M}$ so long as the result remains feasible. We let $\mathcal{M}(\mathcal{E}, \mathbf{C}, \succ^E)$ denote the resulting matching.[3]

The RANDOM-EDGE algorithm selects the order $\succ^E$ uniformly at random. A different set of greedy algorithms consider jobs sequentially. They arise naturally in the context of online matching, where the set of jobs is gradually revealed, and each job must be (irrevocably) scheduled upon arrival. These algorithms are parameterized by an order $\succ^{\mathcal{J}}$ in which jobs will be considered, and a collection of rankings $\succ^{\mathcal{D}} = \{\succ_j^{\mathcal{D}}\}_{j \in \mathcal{J}}$, where $\succ_j^{\mathcal{D}}$ is a ranking over days used when scheduling job $j$. Formally, we define $\text{VI}(\succ^{\mathcal{J}}, \succ^{\mathcal{D}})$ to be the order $\succ^E$ satisfying

$$(j, d) \succ^E (j', d') \Leftrightarrow j \succ^{\mathcal{J}} j' \text{ or } j = j' \text{ and } d \succ_j^{\mathcal{D}} d'.$$

Goel and Tripathi (2012) and Tang et al. (2020) refer to these as "vertex-iterative algorithms," and we choose the letters VI to align with this terminology. Both RANDOM-VERTEX, and RANKING are special cases of vertex-iterative algorithms. RANDOM-VERTEX selects $\succ^{\mathcal{J}}$ and the orders $\succ_j^{\mathcal{D}}$ independently and uniformly at random. RANKING selects the order $\succ^{\mathcal{J}}$ and an order $\succ$ on $\mathcal{D}$ independently and uniformly at random, and sets $\succ_j^{\mathcal{D}} = \succ$ for all $j \in \mathcal{J}$.

[3]Note that the order $\succ^E$ is *non-adaptive*, in that earlier matches do not affect the order in which later edges are considered. This family of algorithms is closely related to the class of "randomized priority algorithms" described by Borodin et al. (2003).

1.2. **Prior Work: Worst Case Analysis.** Most existing analysis assumes one-to-one matching ($C_d = 1$ for all $d$), and studies an algorithm's worst-case approximation ratio: that is, the largest number $\alpha$ such that the expected size of the matching produced by the algorithm is always at least $\alpha$ times the size of a maximum matching.

Dyer and Frieze (1991) show that for RANDOM-EDGE, no guarantee greater than 1/2 is possible. The seminal work of Karp et al. (1990) consider vertex-iterative algorithms a setting where both the scheduling constraints $\mathcal{E}$ and the job arrival order $\succ^{\mathcal{J}}$ are adversarial, and only the orders $\succ_j^{\mathcal{D}}$ can be chosen. They show that if the $\succ_j^{\mathcal{D}}$ are iid and uniformly distributed (the "random" algorithm) then no guarantee greater than 1/2 is possible. They propose instead choosing the $\succ_j^{\mathcal{D}}$ to be *identical* (the "ranking" algorithm), and show that the expected size of the resulting matching is at least $1 - 1/e \approx 0.63$ times the size of the maximum matching.[4]

Subsequent work has assumed that the arrival order $\succ^{\mathcal{J}}$ is drawn uniformly at random, which corresponds to the algorithms RANDOM-VERTEX and RANKING as described in this paper. This enables stronger guarantees. Aronson et al. (1995) prove that RANDOM-VERTEX (which they name "modified random greedy") offers a guarantee of $1/2 + 2.5 \times 10^{-6}$. This was state-of-the-art until Poloczec and Szegedy (2012) proved[5] $1/2 + 1/256$. In recent work, Tang et al. (2020) prove a guarantee of 0.639, and establish an upper-bound of 0.646. For RANKING, Mahdian and Yan (2011) establish a guarantee of 0.696 and Karande et al. (2011) provide an upper bound of 0.727 which has since been improved to 0.724 by Chan et al. (2018).[6] Table 1 summarizes our knowledge of the worst-case performance guarantee for each algorithm.

There is a natural intuition that it is best to start by scheduling low-degree jobs. A variant of RANDOM-VERTEX which does this was proposed by Tinhofer (1984) and subsequently dubbed "MINGREEDY." Frieze et al. (1995) show that this algorithm performs well on random cubic graphs, but somewhat surprisingly, Besser and Poloczek (2017) show that its worst-case guarantee is the trivial one of 1/2.

1.3. **Our Results: Average Case Analysis.** This paper considers the performance of these greedy algorithms when the scheduling constraints $\mathcal{E}$ are random, as defined below.

**Definition 2.** For any $\mathcal{J}, \mathcal{D}$ and $\bar{\mathbf{N}} \in \{0, 1, \ldots, |\mathcal{D}|\}^{\mathcal{J}}$, define $\psi(\bar{\mathbf{N}})$ to be the uniform distribution over edge sets $\mathcal{E}$ for which each $j \in \mathcal{J}$ has exactly $\bar{\mathbf{N}}_j$ neighbors.

Note that one can sample from $\psi(\bar{\mathbf{N}})$ by allowing each $j \in \mathcal{J}$ to independently select $\bar{N}_j$ neighbors in $\mathcal{D}$ uniformly at random. This family of bipartite random graphs has previously been studied in the context of cuckoo hashing (Dietzfelbinger et al. 2010, Fountoulakis and Panagiotou 2012, Frieze et al. 2018).

Our first result establishes that for this family of random graphs, the size of the matching generated by RANDOM-VERTEX stochastically dominates the size of the matching generated by RANDOM-EDGE.

**Theorem 1.** *Fix $\mathcal{J}, \mathcal{D}$. For any $\mathbf{C} \in \mathbb{N}^{\mathcal{D}}$ and $\bar{\mathbf{N}} \in \{0, 1, \ldots |\mathcal{D}|\}^{\mathcal{J}}$, if $\mathcal{E}$ is drawn from $\psi(\bar{\mathbf{N}})$, then for all $k \in \mathbb{N}$,*

$$\mathbb{P}(|\mathcal{M}(\mathcal{E}, \mathbf{C}, \text{RANDOM-VERTEX})| \geq k) \geq \mathbb{P}(|\mathcal{M}(\mathcal{E}, \mathbf{C}, \text{RANDOM-EDGE})| \geq k).$$

[4]Although the results claimed by Karp et al. (1990) are correct, Goel and Mehta (2008) observed and corrected an error in their original analysis.

[5]Chan et al. (2018) comment that there are several gaps in their proof.

[6]It is possible to generalize RANDOM-VERTEX and RANKING to general (non-bipartite) graphs. Of course, this results in lower worst-case performance. Goel and Tripathi (2012) claimed a guarantee of 0.56 for RANKING, but retracted the paper after errors were found. Chan et al. (2018) establish a guarantee of 0.523 for RANKING, and recent work by Tang et al. (2020) establishes a guarantee of 0.531 for both RANDOM-VERTEX and RANKING.

Note that this immediately implies the corresponding result when the values $\bar{\mathbf{N}}_j$ are drawn from an arbitrary joint distribution (i.e. the case of bipartite Erdös-Renyi graphs). Sections A.1 and A.2 establish that this result continues to hold in extensions where jobs have different "sizes" and some days are more likely to be feasible than others.

Our second result establishes that when only one job can be scheduled on each day, all vertex-iterative algorithms that consider jobs in the same order have equivalent performance: the preferences of individual jobs do not affect the distribution of the size of the resulting matching. With some abuse of notation, we write $j \in \mathcal{M}$ as shorthand for "$(j, d) \in \mathcal{M}$ for some $d \in \mathcal{D}$."

**Theorem 2.** *Fix $\mathcal{J}, \mathcal{D}, \bar{\mathbf{N}}$. Let $\succ^{\mathcal{J}}$ be an order on $\mathcal{J}$, and let $\succ^{\mathcal{D}}$ and $\tilde{\succ}^{D}$ be two profiles of orders on $\mathcal{D}$. If $C_d = 1$ for all $d \in \mathcal{D}$, and $G$ is drawn from $\psi(\bar{\mathbf{N}})$, then for all $j \in \mathcal{J}$,*

$$\mathbb{P}(j \in \mathcal{M}(\mathcal{E}, \mathrm{VI}(\succ^{\mathcal{J}}, \succ^{\mathcal{D}}))) = \mathbb{P}(j \in \mathcal{M}(\mathcal{E}, \mathrm{VI}(\succ^{\mathcal{J}}, \tilde{\succ}^{D}))).$$

A corollary of Theorem 2 is that when each day has capacity $C_d = 1$ (as much of the literature assumes), RANKING and RANDOM-VERTEX have identical performance. By Theorem 1, both result in a stochastically larger matching than RANDOM-EDGE. When some days have capacities $C_d > 1$, there are examples in which RANDOM-VERTEX outperforms RANKING, and vice versa (see Appendix B for examples). We conjecture that when all days have identical capacity, RANDOM-VERTEX always weakly outperforms RANKING.

We are not the first paper to conduct average-case analysis of these greedy matching algorithms. Dyer et al. (1993) study the asymptotic performance of RANDOM-EDGE and RANDOM-VERTEX (which they refer to as "GREEDY" and "MODIFIED GREEDY," respectively) on two families: sparse Erdös-Renyi random graphs, and random labeled trees. For both families, they show that as the number of vertices grows large, the expected matching size is larger under RANDOM-VERTEX than under RANDOM-EDGE. Although our results are technically incomparable because we study bipartite random graphs, Theorem 1 is "stronger" than this result in several ways.

- It holds for graphs of arbitrary size, rather than only asymptotically.
- It allows vertices in $\mathcal{D}$ to have arbitrary capacities, rather than assuming $C_d = 1\ \forall d$.
- It allows for an arbitrary degree distribution among vertices in $\mathcal{J}$, rather than the binomial distribution that arises in Erdös-Renyi random graphs.
- As discussed in Appendix A, minor modifications to the proof of Theorem 1 can accommodate heterogeneity in the "size" of vertices in $\mathcal{J}$ and probability of feasibility for vertices in $\mathcal{D}$.

To our knowledge, the only other paper that attempts a non-asymptotic average-case comparison of RANDOM-EDGE and RANDOM-VERTEX is Tinhofer (1984). That analysis is restricted to the case of Erdös-Renyi random graphs, and a comparison between the algorithms is provided only when each edge is present in $\mathcal{E}$ with probability exceeding 1/2.

Meanwhile, Lemma 4 in Mastin and Jaillet (2018) notes that on Erdös-Renyi random graphs, the performances of RANKING and RANDOM-VERTEX are identical. Theorem 2 can be seen as a generalization of their result along two dimensions:

- It allows for an arbitrary degree distribution among vertices in $\mathcal{J}$, rather than the binomial distribution that arises in Erdös-Renyi random graphs.
- It holds for arbitrary vertex-iterative algorithms, rather than assuming that $\succ^{\mathcal{J}}$ and $\succ^{\mathcal{D}}_j$ are drawn uniformly at random.

Although the proof of this result is straightforward, we believe it to be an important observation that the equivalence relies only on randomness of the edge set $\mathcal{E}$, and not on randomness of the algorithms themselves.

## 2. Proof of Theorem 1

The key intuition underlying the proof of Theorem 1 is that it is best to start by scheduling jobs with few feasible days, because jobs with many feasible days can likely be scheduled later. RANDOM-EDGE does the *opposite*: at each step, jobs with more feasible days are more likely to be scheduled. Meanwhile, RANDOM-VERTEX schedules jobs in a random order, and thus is more likely than RANDOM-EDGE to schedule low-degree jobs.

To formalize this idea, Section 2.1 describes Markov chains corresponding to procedures RANDOM-EDGE and RANDOM-VERTEX. In each Markov chain, the matching $\mathcal{M}$ is constructed iteratively, starting with $\mathcal{M}_0 = \emptyset$ and adding one edge at a time. Rather than revealing the random edges $\mathcal{E}$ initially, these chains start with $\mathcal{E}_0 = \emptyset$ and reveal an edge $e = (j, d) \in \mathcal{E}$ only when either i) $j$ is matched, or ii) $d$ reaches its capacity. Potential edges between unassigned jobs and days with excess capacity remain unobserved. The size of the matching generated by each procedure corresponds to the number of steps before the full edge set $\mathcal{E}$ is revealed (implying that no more matches can be formed).

Section 2.2 couples these chains such that the number of unrevealed edges under RANDOM-VERTEX is always weakly larger than the corresponding number under RANDOM-EDGE. Whenever the number of jobs with at least $k$ unrevealed edges is identical in the two chains, the coupling ensures that if the next job assigned under RANDOM-VERTEX has at least $k$ unrevealed edges, then so does the next job assigned under RANDOM-EDGE. Figure 2 provides an illustration of each chain, and the coupling between them.

### 2.1. Proof Step 1: Markov Chain Description.

For both RANDOM-VERTEX and RANDOM-EDGE, we start with an empty edge set and matching $\mathcal{E}_0 = \mathcal{M}_0 = \emptyset$. At each step $t$, we add one edge between an unscheduled job and a day with idle capacity to the matching $\mathcal{M}_t$ (if no such edge exists, then our procedure has finished). We also reveal any edges involving matched jobs or days with no remaining capacity and add these to the edge set $\mathcal{E}_t$. This ensures that unrevealed edges are precisely those that can be feasibly added to $\mathcal{M}_t$. In other words, for any $e = (j, d)$,

$$\mathcal{M}_t \cup \{e\} \text{ is feasible} \Leftrightarrow e \in \mathcal{E} \backslash \mathcal{E}_t.$$

We now describe the procedures by which edges are added to $\mathcal{M}_t$ under RANDOM-VERTEX and RANDOM-EDGE. We first offer a description in English, and then provide corresponding mathematical expressions. For each procedure, step $t + 1$ consists of four stages:

- V1 Reveal the job $j_{t+1}$ which ranks highest (according to $\succ^{\mathcal{J}}$) among unassigned jobs for which at least one feasible day has idle capacity.
- V2 Observe the days $D_{t+1}$ that are feasible for $j_{t+1}$ and have idle capacity.
- V3 Assign $j_{t+1}$ to a random day $d_{t+1} \in D_{t+1}$.
- V4 If day $d_{t+1}$ no longer has idle capacity, then for each unassigned job $j$, observe whether $d_{t+1}$ was feasible for $j$.

- E1 Reveal the job $j_{t+1}$ which has the highest-ranking edge in $\mathcal{E} \backslash \mathcal{E}_t$ (according to $\succ^{E}$).
- E2 Observe the days $D_{t+1}$ that are feasible for $j_{t+1}$ and have idle capacity.
- E3 Assign $j_{t+1}$ to a random day $d_{t+1} \in D_{t+1}$.
- E4 If day $d_{t+1}$ no longer has idle capacity, then for each unassigned job $j$, observe whether $d_{t+1}$ was feasible for $j$.

We now provide equations corresponding to each of these chains. Fix a number of feasible days $\bar{N}_j$ for each job $j$ and capacities $C_d$ for each day $d$. Given a set of edges $\mathcal{E}_t$ and a feasible matching

$\mathcal{M}_t \subseteq \mathcal{E}_t$, define

$$\begin{aligned} I_d(\mathcal{M}_t) &= C_d - |\{e \in \mathcal{M}_t : e = (j,d) \text{ for some } j\}| && \text{Idle capacity on day } d. \\ N_j(\mathcal{E}_t) &= \bar{N}_j - |\{e \in \mathcal{E}_t : e = (j,d) \text{ for some } d\}| && \text{Number of unknown feasible days for job } j. \\ F_k(\mathcal{E}_t) &= |\{j : N_j(\mathcal{E}_t) = k\}| && \text{Number of jobs with } k \text{ unknown feasible days.} \end{aligned}$$

The stages V1-V4 above can be expressed as follows:

V1 **Reveal the next job to be assigned.** Select $j_{t+1}$ with

$$\mathbb{P}(j_{t+1} = j | \mathcal{M}_t, \mathcal{E}_t) = \frac{\mathbf{1}(N_j(\mathcal{E}_t) > 0))}{\sum_{j'} \mathbf{1}(N_{j'}(\mathcal{E}_t) > 0)}. \tag{1}$$

V2 **Reveal feasible days for $j_{t+1}$.** Select $D_{t+1} \subseteq \{d : I_d(\mathcal{M}_t) > 0\}$ uniformly at random among subsets of size $N_{j_{t+1}}(\mathcal{M}_t)$, and define

$$\mathcal{A}_{t+1} = \{(j_{t+1}, d) : d \in D_{t+1}\}. \tag{2}$$

V3 **Assign $j_{t+1}$ to a feasible day.** Select $d_{t+1}$ uniformly at random from $D_{t+1}$, and set

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup (j_{t+1}, d_{t+1}). \tag{3}$$

Note that steps V2 and V3 imply that $d_{t+1}$ is selected uniformly at random from days with idle capacity. That is,

$$\mathbb{P}(d_{t+1} = d | \mathcal{M}_t, \mathcal{E}_t, j_{t+1}) = \frac{\mathbf{1}(I_d(\mathcal{M}_t) > 0)}{|\{d : I_d(\mathcal{M}_t) > 0\}|}. \tag{4}$$

V4 **If $d_{t+1}$ has no remaining capacity, reveal neighbors of $d_{t+1}$ in $\mathcal{E}$.** Draw independent binary random variables $\{B_{(t+1)j}\}_{j \in \mathcal{J}}$, with

$$\mathbb{P}(B_{(t+1)j} = 1 | \mathcal{E}_t, \mathcal{M}_t, j_{t+1}, D_{t+1}, d_{t+1}) = \frac{N_j(\mathcal{E}_t)}{|\{d : I_d(\mathcal{M}_t) > 0\}|} \mathbf{1}(j \neq j_{t+1}) \mathbf{1}(I_{d_{t+1}}(\mathcal{M}_{t+1}) = 0). \tag{5}$$

Set

$$\mathcal{B}_{t+1} = \{(j, d_{t+1}) : B_{(t+1)j} = 1\} \tag{6}$$

$$\mathcal{E}_{t+1} = \mathcal{E}_t \cup \mathcal{A}_{t+1} \cup \mathcal{B}_{t+1}. \tag{7}$$

The stage E1 corresponds to the following:

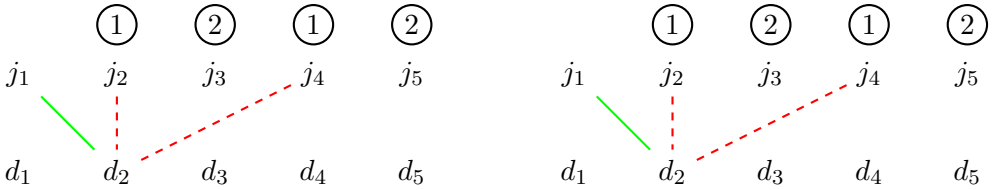
E1 Select $j_{t+1}$ with

$$\mathbb{P}(j_{t+1} = j | \mathcal{M}_t, \mathcal{E}_t) = \frac{N_j(\mathcal{E}_t)}{\sum_{j'} N_{j'}(\mathcal{E}_t)}. \tag{8}$$

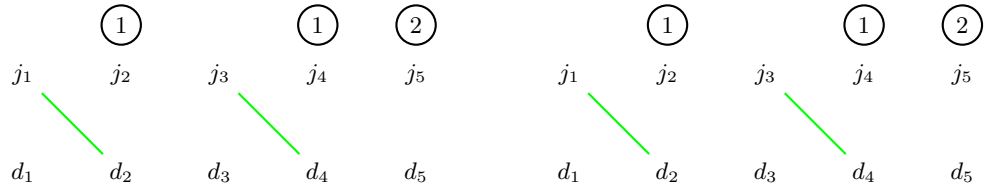
Meanwhile, stages E2-E4 are identical to stages V2-V4. In other words, the only difference between the procedures is that step V1 selects an unassigned job uniformly at random among those with at least one feasible day remaining, whereas step E1 selects among these jobs *in proportion to the number of feasible days remaining.*

Both procedures terminate when no more edges can be feasibly added – that is, when $F_k(\mathcal{E}_t) = 0$ for all $k \geq 1$.

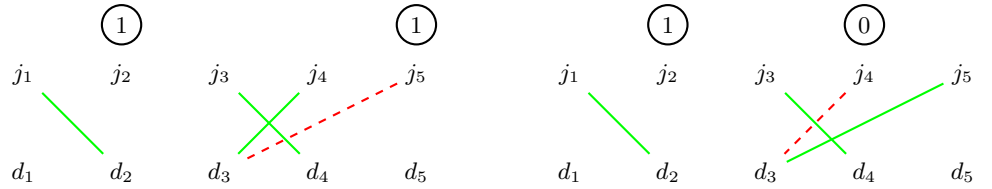
**RANDOM-VERTEX** **RANDOM-EDGE**



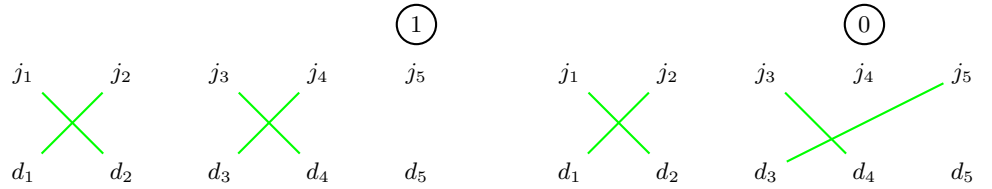

A match is formed between $j_1$ and $d_2$. Throughout, (9) ensures that the set of assigned days is identical for the two chains. Because (13) is tight, the coupling ensures that the revealed (red dashed) edges for the two chains coincide. It is revealed that $d_2$ was also feasible for $j_2$ and $j_4$.




The next match under RANDOM-VERTEX involves $j_3$, for which two remaining days are feasible. Because (10) is tight, the coupling ensures that the next match under RANDOM-EDGE also involves a job for which two remaining days are feasible. Because (13) is tight, the (empty) sets of revealed edges coincide. Jobs $j_2$ and $j_4$ are more likely to go unassigned than $j_5$, for which two of $\{d_1, d_3, d_5\}$ are feasible.




Because (10) is tight, the next job assigned under RANDOM-EDGE has weakly more remaining feasible days (see Lemma 2). The next match under RANDOM-VERTEX involves $j_4$, whereas under RANDOM-EDGE it involves $j_5$. For the first time, $\phi$ is not the identity: it permutes $j_4$ and $j_5$. Job $j_4$ can no longer be feasibly assigned under RANDOM-EDGE.




RANDOM-EDGE must match $j_2$. RANDOM-VERTEX randomly selects which of $j_2$ and $j_5$ to match. Job $j_2$ is assigned to $d_1$, and it is revealed that $d_1$ was not feasible for $j_5$. RANDOM-EDGE has completed, whereas RANDOM-VERTEX will continue for one more step.

FIGURE 2. A visualization of the chains corresponding to RANDOM-VERTEX and RANDOM-EDGE on an instance with five jobs and five days. Each job has two feasible days, and only one job can be scheduled for each day. In each step, an unscheduled job is assigned to one remaining feasible day (solid green line), and the set of other jobs that were feasible for this day is revealed (dashed red lines). The numbers above each job denote the number of remaining feasible days. Under RANDOM-VERTEX, jobs are selected in uniformly at random, whereas under RANDOM-EDGE, they are selected in proportion to the number of remaining feasible days.

2.2. **Proof Step 2: Coupling.** In what follows, we use the superscript $V$ to denote the chain corresponding to RANDOM-VERTEX, and $E$ to denote the chain corresponding to RANDOM-EDGE. In other words, $\mathcal{M}_t^V, \mathcal{E}_t^V$ refer to the (random) edges and matching generated after $t$ steps of V1-V4, and $\mathcal{M}_t^E, \mathcal{E}_t^E$ to the (random) edges and matching generated after $t$ steps of E1-E4. For $X \in \{V, E\}$, let $j_{t+1}^X$ be the job assigned at step $t+1$, let $D_{t+1}^X$ be the days to which this job could feasibly be assigned, and let $d_{t+1}^X$ be the day to which this job is actually assigned. Let $\tau^X$ be the time satisfying $F_k(\mathcal{E}_{\tau^X}^X) = 0$ for all $k \geq 1$. In other words, $\tau^X$ is the time at which it is no longer possible to add feasible edges to $\mathcal{M}_t^X$. The following Lemma immediately implies Theorem 1.

**Lemma 1.** *There exists a coupling of $(\mathcal{M}_t^V, \mathcal{E}_t^V)$ and $(\mathcal{M}_t^E, \mathcal{E}_t^E)$ such that for all $t \leq \tau^E$ and all $k \in \{1, \ldots, |\mathcal{D}|\}$ the following hold:*

$$I(\mathcal{M}_t^V) = I(\mathcal{M}_t^E). \tag{9}$$

$$\sum_{i \geq k} F_i(\mathcal{E}_t^V) \geq \sum_{i \geq k} F_i(\mathcal{E}_t^E). \tag{10}$$

*Proof.* First, (4) implies that if $I(\mathcal{M}_t^V) = I(\mathcal{M}_t^E)$, then regardless of the realizations $j_{t+1}^V, j_{t+1}^E$, the chains can be coupled so that $d_{t+1}^V = d_{t+1}^E$, and therefore (3) implies $I(\mathcal{M}_{t+1}^V) = I(\mathcal{M}_{t+1}^E)$. In light of this fact, we write $d_{t+1}$ in place of $d_{t+1}^V = d_{t+1}^E$, and $I(\mathcal{M}_t)$ in place of $I(\mathcal{M}_t^V) = I(\mathcal{M}_t^E)$.

We now turn to (10). For $X \in \{V, E\}$ we define

$$\mathcal{A}_{t+1}^X = \{(j_{t+1}^X, d) : d \in D_{t+1}^X\} \tag{11}$$

Lemma 2 states that it is possible to couple $\mathcal{A}_{t+1}^V, \mathcal{A}_{t+1}^E$ such that

$$\sum_{i \geq k} F_i(\mathcal{E}_t^V \cup \mathcal{A}_{t+1}^V) \geq \sum_{i \geq k} F_i(\mathcal{E}_t^E \cup \mathcal{A}_{t+1}^V). \tag{12}$$

That is, an analog of (10) holds after stages V3 and E3. If $I_{d_{t+1}}(\mathcal{M}_{t+1}) > 0$, then by (5) and (6) $\mathcal{B}_{t+1}^V = \mathcal{B}_{t+1}^E = \emptyset$, so for $X \in \{V, E\}$, (7) implies that $\mathcal{E}_{t+1}^X = \mathcal{E}_t^X \cup \mathcal{A}_{t+1}^X$, and there is nothing more to do.

Otherwise, we must couple $B_{t+1}^V$ and $B_{t+1}^E$. It follows from (12) that there exists a permutation $\phi$ on $\mathcal{J}$ such that $\phi(j_{t+1}^V) = j_{t+1}^E$, and for all $j$,

$$N_j(\mathcal{E}_t^V \cup \mathcal{A}_{t+1}^V) \geq N_{\phi(j)}(\mathcal{E}_t^E \cup \mathcal{A}_{t+1}^E). \tag{13}$$

For each $j$, if equality holds in (13), couple $B_{(t+1)j}^V$ and $B_{(t+1)\phi(j)}^E$ such that $B_{(t+1)j}^V = B_{(t+1)\phi(j)}^E$. If the inequality in (13) is strict, generate $B_{(t+1)j}^V$ and $B_{(t+1)\phi(j)}^E$ independently. This ensures that

$$N_j(\mathcal{E}_{t+1}^V) = N_j(\mathcal{E}_t^V \cup \mathcal{A}_{t+1}^V \cup \mathcal{B}_{t+1}^V) \geq N_{\phi(j)}(\mathcal{E}_t^E \cup \mathcal{A}_{t+1}^E \cup \mathcal{B}_{t+1}^E) = N_{\phi(j)}(\mathcal{E}_{t+1}^E) \tag{14}$$

for all $j$, and therefore that (10) holds.

□

**Lemma 2.** *If (10) holds, then it is possible to couple $\mathcal{A}_{t+1}^V, \mathcal{A}_{t+1}^E$ such that for $k \in \{1, \ldots, |\mathcal{D}|\}$,*

$$\sum_{i \geq k} F_i(\mathcal{E}_t^V \cup \mathcal{A}_{t+1}^V) \geq \sum_{i \geq k} F_i(\mathcal{E}_t^E \cup \mathcal{A}_{t+1}^E).$$

*Proof.* For $X \in \{V, E\}$ define $K_{t+1}^X = N_{j_{t+1}}(\mathcal{E}_t^X)$ to be the number of feasible days for job $j_{t+1}$ that still have idle capacity. Then the definition of $\mathcal{A}_{t+1}^X$ in (2) implies that for $k \in \{1, \ldots, |\mathcal{D}|\}$,

$$\sum_{i \geq k} F_i(\mathcal{E}_t^X \cup \mathcal{A}_{t+1}^X) = \sum_{i \geq k} F_i(\mathcal{E}_t^X) - \mathbf{1}(K_{t+1}^X \geq k).$$

Thus, it is enough to show that if $\sum_{i\geq k} F_i(\mathcal{E}_t^V) = \sum_{i\geq k} F_i(\mathcal{E}_t^E)$, it is possible to couple $K_{t+1}^V$ and $K_{t+1}^E$ such that $K_{t+1}^V \geq k$ implies that $K_{t+1}^E \geq k$. In other words, we must show that

$$\mathbb{P}(K_{t+1}^E \geq k | \mathcal{M}_t^E, \mathcal{E}_t^E) \geq \mathbb{P}(K_{t+1}^V \geq k | \mathcal{M}_t^V, \mathcal{E}_t^V).$$

Note that (1) implies that for $i \geq 1$,

$$\mathbb{P}(K_{t+1}^V = i | \mathcal{M}_t^V, \mathcal{E}_t^V) = \frac{F_i(\mathcal{E}_t^V)}{\sum_{j\geq 1} F_j(\mathcal{E}_t^V)}.$$

Analogously, (8) implies that

$$\mathbb{P}(K_{t+1}^E = i | \mathcal{M}_t^E, \mathcal{E}_t^E) = \frac{iF_i(\mathcal{E}_t^E)}{\sum_{j\geq 1} jF_j(\mathcal{E}_t^E)}.$$

It follows that if $\sum_{i\geq k} F_i(\mathcal{E}_t^V) = \sum_{i\geq k} F_i(\mathcal{E}_t^E)$, then

$$\begin{aligned}
\mathbb{P}(K_{t+1}^V \geq k | \mathcal{M}_t^V, \mathcal{E}_t^V) &= \frac{\sum_{i\geq k} F_i(\mathcal{E}_t^V)}{\sum_{i\geq 1} F_i(\mathcal{E}_t^V)} \\
&\leq \frac{\sum_{i\geq k} F_i(\mathcal{E}_t^V)}{\sum_{i\geq 1} F_i(\mathcal{E}_t^E)} \\
&= \frac{\sum_{i\geq k} F_i(\mathcal{E}_t^E)}{\sum_{i\geq 1} F_i(\mathcal{E}_t^E)} \\
&\leq \frac{\sum_{i\geq k} iF_i(\mathcal{E}_t^E)}{\sum_{i\geq 1} iF_i(\mathcal{E}_t^E)} \\
&= \mathbb{P}(K_{t+1}^E \geq k | \mathcal{M}_t^E, \mathcal{E}_t^E).
\end{aligned}$$

The first inequality follows from (10), and the final inequality follows because

$$\begin{aligned}
\frac{\sum_{i\geq k} F_i(\mathcal{E}_t^E)}{\sum_{i\geq 1} F_i(\mathcal{E}_t^E)} &= \left(1 + \sum_{i<k} kF_i(\mathcal{E}_t^E) / \sum_{i\geq k} kF_i(\mathcal{E}_t^E)\right)^{-1} \\
&\leq \left(1 + \sum_{i<k} iF_i(\mathcal{E}_t^E) / \sum_{i\geq k} iF_i(\mathcal{E}_t^E)\right)^{-1} = \frac{\sum_{i\geq k} iF_i(\mathcal{E}_t^E)}{\sum_{i\geq 1} iF_i(\mathcal{E}_t^E)}.
\end{aligned}$$

□

## 3. Proof of Theorem 2

This section proves Theorem 2, which states that when $C_d = 1$ for all $d$, the probability that a given job $j$ matches under $\text{VI}(\succ^{\mathcal{J}}, \succ^{\mathcal{D}})$ does not depend on $\succ^{\mathcal{D}}$. As before, we iteratively construct $\mathcal{E}_t$ and $\mathcal{M}_t$. Step $t+1$ consists of the following four stages:

SD1 Let $j_{t+1}$ be the first job (according to $\succ^{\mathcal{J}}$) that is unmatched in $\mathcal{M}_t$ and has $N_j(\mathcal{E}_t) > 0$.

SD2 Select $D_{t+1} \subseteq \{d : I_d(\mathcal{M}_t) > 0\}$ uniformly at random among subsets of size $N_{j_{t+1}}(\mathcal{M}_t)$. Define

$$\mathcal{A}_{t+1} = \{(j_{t+1}, d) : d \in D_{t+1}\}.$$

SD3 Let $d_{t+1}$ be the first element in $D_{t+1}$ (according to $\succ_{j_{t+1}}$), and set

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup (j_{t+1}, d_{t+1}).$$

SD4 Draw independent binary random variables $\{B_{(t+1)j}\}_{j\in\mathcal{J}}$, with

$$\mathbb{P}(B_{(t+1)j}=1|\mathcal{E}_t,\mathcal{M}_t,D_{t+1})=\frac{N_j(\mathcal{E}_t)}{|\{d:I_d(\mathcal{M}_t)>0\}|}\mathbf{1}(j\neq j_{t+1})\mathbf{1}(I_{d_{t+1}}(\mathcal{M}_{t+1})=0). \tag{15}$$

Let $\mathcal{B}_{t+1}=\{(j,d_{t+1}):B_{(t+1)j}=1\}$, and set

$$\mathcal{E}_{t+1}=\mathcal{E}_t\cup\mathcal{A}_{t+1}\cup\mathcal{B}_{t+1}.$$

Stages SD2 and SD4 are identical to V2 and V4, respectively. The only differences between this Markov chain and that given in V1-V4 are that in steps SD1 and SD3, job $j_{t+1}$ and day $d_{t+1}$ are determined by $\succ^{\mathcal{J}}$ and $\succ_{j_{t+1}}$, and thus are deterministic given $\mathcal{M}_t,\mathcal{E}_t$, and $D_{t+1}$.

Let $\mathcal{M}_t,\mathcal{E}_t$ be the (random) matching and edge set from VI($\succ^{\mathcal{J}},\succ^{\mathcal{D}}$) and $\mathcal{M}'_t,\mathcal{E}'_t$ be the corresponding matching and edge set under VI($\succ^{\mathcal{J}},\tilde{\succ}^{D}$). We claim that the chains can be coupled such that for all $j,t$,

$$\{j:(j,d)\in\mathcal{M}_t\text{ for some }d\}=\{j:(j,d)\in\mathcal{M}'_t\text{ for some }d\} \tag{16}$$

$$N_j(\mathcal{E}_t)=N_j(\mathcal{E}'_t), \tag{17}$$

from which Theorem 2 follows. If (17) holds at $t$, it follows immediately from SD1 that

$$j_{t+1}=j'_{t+1}, \tag{18}$$

and therefore (16) holds for $\mathcal{M}_{t+1}$ and $\mathcal{M}'_{t+1}$. Furthermore, because $C_d=1$ for all $d$, each round reduces the set of available days by one, so

$$|\{d:I_d(\mathcal{M}_t)>0\}|=\left|\{d:I_d(\mathcal{M}'_t)>0\}\right|=|\mathcal{D}|-t. \tag{19}$$

It follows from (15), (17), (18) and (19) that for all $j$,

$$\mathbb{P}(B_{(t+1)j}=1|\mathcal{E}_t,\mathcal{M}_t,D_{t+1})=\mathbb{P}(B'_{(t+1)j}=1|\mathcal{E}'_t,\mathcal{M}'_t,D'_{t+1}),$$

so we can ensure that $B_{(t+1)j}=B'_{(t+1)j}$, and therefore that $N_j(\mathcal{E}_{t+1})=N_j(\mathcal{E}'_{t+1})$.

## 4. Discussion

Variants of the greedy algorithms studied in this paper arise frequently in practice. In many contexts, for example, "jobs" arrive dynamically and must be assigned immediately. The most well-studied example is online advertising, where impressions are assigned to advertisers in real time: see for example Mehta et al. (2007), Goel and Mehta (2008), Feldman et al. (2009), Aggarwal et al. (2011), and a helpful survey by Mehta (2013). As another example, many wilderness areas have a limited number of camping permits for each day, which are assigned dynamically: on a first-come-first-served basis, prospective campers can select among days for which permits remain. This system corresponds to a vertex-iterative algorithm in which $\succ^{\mathcal{J}}$ encodes campers' order of arrival, and $\succ^{\mathcal{D}}_j$ encodes the preferences of camper $j$.

Even when items are allocated in a single round, it is common to use a "random serial dictatorship," which is a vertex-iterative algorithm where the order $\succ^{\mathcal{J}}$ is selected uniformly at random.[7] In addition to its procedural fairness, this approach is simple to describe and implement, and incentivizes applicants to submit their preferences and constraints truthfully. By contrast, algorithms which select maximum matchings have none of these features. The algorithms RANDOM-VERTEX and RANKING can be thought of as a random serial dictatorship in which applicants have independent or identical preferences over days, respectively. Meanwhile, the procedure RANDOM-EDGE arises if a new lottery number is assigned to each *application* rather than each *applicant*.

To be clear, our results are primarily intended to contribute to the academic literature, rather than to offer immediate policy guidance. Real-world graphs are likely neither selected adversarially,

[7]This is in fact how permits for the popular Half Dome hike in Yosemite are allocated – see `https://www.nps.gov/yose/planyourvisit/hdpermits.htm`.

nor drawn uniformly at random. However, to the extent that academic work does guide practice, we believe that average-case analysis is a useful complement to the prevailing paradigm of worst-case analysis.

In some cases, such as the comparison between RANDOM-EDGE and RANDOM-VERTEX in Theorem 1, average-case analysis confirms the story told by worst-case guarantees. In others, it does not: while RANKING offers a better worst-case guarantee than RANDOM-VERTEX, Theorem 2 establishes that they have identical average-case performance for one-to-one matching. Although this result is easy to prove, we believe that it is conceptually important, given the prevailing wisdom that RANKING is a "better" algorithm. In fact, we conjecture that RANDOM-VERTEX has better average performance than RANKING in many-to-one matching when days have identical capacities.

Another example where worst-case analysis may be misleading is the case of the MINGREEDY algorithm, which considers jobs in increasing order of their degree. Although this algorithm guarantees only the trivial factor of 1/2 (Besser and Poloczek 2017), we conjecture that coupling arguments similar to those used in this paper could establish that on average, it outperforms RANDOM-VERTEX for any degree distribution among jobs.

To get closer to practical application, many bells and whistles can be added to our basic model. For example, Appendix A extends the conclusions of Theorem 1 to settings where jobs have different sizes and some days are more likely to be feasible than others. Our results also have implications for weighted matching. If edge weights are drawn iid, then RANDOM-EDGE corresponds to the natural greedy procedure which sequentially adds the highest-weight edge that maintains feasibility. This guarantees a matching with weight at least half of optimal. Meanwhile, RANDOM-VERTEX corresponds to a vertex-iterative procedure that considers jobs in random order and selects the highest-weight feasible edge. Although this approach does not offer any worst-case guarantee, Theorem 1 implies that if the distribution of edge weights is sufficiently concentrated, it will on average produce a higher-weight matching than greedy edge selection.

Although Theorem 1 states that RANDOM-VERTEX produces a larger matching than RANDOM-EDGE, it is silent on the magnitude of the difference. This of course depends on the graph parameters. For example, the difference should be small if all jobs have sufficiently high degree, or if there is a large imbalance between the sides (in which case the number of matches should be close to the minimum capacity of the two sides). The differential equation method from Wormald (1995, 1999) makes it possible to derive expressions for the asymptotic matching size in a regime where the $N_j$ and $C_d$ are drawn iid, $|\mathcal{J}| / |\mathcal{D}|$ is fixed and $|\mathcal{D}| \to \infty$. Although a comprehensive study of asymptotic match sizes is not our focus, Appendix C provides expressions for the asymptotic performance of RANDOM-VERTEX and RANDOM-EDGE in sparse bipartite Erdös-Renyi random graphs. These expressions suggest that the difference between RANDOM-VERTEX and RANDOM-EDGE is fairly small; we leave as future work a more complete exploration of the difference for other degree distributions.

## References


Aggarwal G, Goel G, Karande C, Mehta A (2011) Online vertex-weighted bipartite matching and single-bid budgeted allocations. *Symposium on Discrete Algorithms (SODA)* 1253–1264.

Aronson J, Dyer M, Frieze A, Suen S (1995) Randomized greedy matching ii. *Random Structures & Algorithms* 6(1):55–73.

Besser B, Poloczek M (2017) Greedy matching: Guarantees and limitations. *Algorithmica* 77:201–234.

Borodin A, Nielsen M, Rackoff C (2003) (incremental) priority algorithms. *Algorithmica* 37:295–326.

Chan THH, Chen F, Wu X, Zhao Z (2018) Ranking on arbitrary graphs: Rematch via continuous lp with monotone and boundary condition constraints. *SIAM Journal on Computing* 47(4):1529–1546.

Dietzfelbinger M, Goerdt A, Mitzenmacher M, Montanari A, Pagh R, Rink M (2010) Tight thresholds for cuckoo hashing via xorsat. Abramsky S, Gavoille C, Kirchner C, Meyer auf der Heide F, Spirakis P, eds., *Automata, Languages and Programming*, volume 6198 of *Lecture Notes in Computer Science*, 213–225 (Springer Berlin Heidelberg), ISBN 978-3-642-14164-5, URL `http://dx.doi.org/10.1007/978-3-642-14165-2_19`.

Dyer M, Frieze A (1991) Randomized greedy matching. *Random Structures & Algorithms* 2(1):29–45.

Dyer M, Frieze A, Pittel B (1993) The average performance of the greedy matching algorithm. *The Annals of Applied Probability* 3(2):pp. 526–552, ISSN 10505164, URL `http://www.jstor.org/stable/2959644`.

Feldman J, Mehta A, Mirrokni V, Muthukrishnan S (2009) Online stochastic matching: Beating 1-1/e. *Proceedings of the 50th Annual IEEE Symposium on Foundations of Computer Science* 117–126.

Fountoulakis N, Panagiotou K (2012) Sharp load thresholds for cuckoo hashing. *Random Structures & Algorithms* 41(3):306–333, ISSN 1098-2418, URL `http://dx.doi.org/10.1002/rsa.20426`.

Frieze A, Melsted P, Mitzenmacher M (2018) An analysis of random-walk cuckoo hashing.

Frieze A, Radcliffe A, Suen S (1995) Analysis of a simple greedy matching algorithm on random cubic graphs. *Combinatorics, Probability and Computing* 4:47–66.

Goel G, Mehta A (2008) Online budgeted matching in random input models with applications to adwords. *Proceedings of the 19th Annual ACM-SIAM Symposium on Discrete Algorithms* 982–991.

Goel G, Tripathi P (2012) Matching with our eyes closed. *Symposium on Fondations of Computer Science (FOCS)* 718–727.

Karande C, Mehta A, Tripathi P (2011) Online bipartite matching with unknown distributions. *ACM Symposium on Theory of Computing (STOC)*.

Karp RM, Vazirani UV, Vazirani VV (1990) An optimal algorithm for on-line bipartite matching. *Proceedings of the twenty-second annual ACM symposium on theory of computing* 352–358.

Mahdian M, Yan Q (2011) Online bipartite matching with random arrivals: An approach based on strongly factor-revealing lps. *ACM Symposium on Theory of Computing (STOC)*.

Mastin A, Jaillet P (2018) Greedy online bipartite matching on random graphs. *arXiv preprint arXiv:1307.2536* .

Mehta A (2013) Online matching and ad allocation. *Foundations and Trends in Theoretical Computer Science* 8(4):265–368.

Mehta A, Saberi A, Vazirani U, Vazirani V (2007) Adwords and generalized online matching. *J. ACM* 54(5).

Poloczec M, Szegedy M (2012) Randomized greedy algorithms for the maximum matching problem with new analysis. *Symposium on Fondations of Computer Science (FOCS)* .

Tang ZG, Wu X, Zhang Y (2020) Towards a better understanding of randomized greedy matching. *Symposium on Theory of Computing (STOC)* 1097–1110.

Tinhofer G (1984) A probabilistic analysis of some greedy cardinality matching algorithms. *Annals of Operations Research* 1(3):239–254.

Wormald NC (1995) Differential equations for random processes and random graphs. *The Annals of Applied Probability* 5(4):pp. 1217–1235, ISSN 10505164, URL `http://www.jstor.org/stable/2245111`.

Wormald NC (1999) The differential equation method for random graph processes and greedy algorithms. *Lectures on approximation and randomized algorithms*, 73–155.

## Appendix A. Extensions of Theorem 1

This appendix extends the conclusions of Theorem 1 to more general settings.

A.1. **Job Sizes.** In our first extension, jobs have different "sizes" $S_j$. For example, $C_d$ might represent the number of hours that developer $d$ can work, and $S_j$ the number of hours that job $j$ will take to complete. Alternately, in the context of hiking permits or theater performances, $C_d$ might represent the number of permits or tickets available on day $d$, and $S_j$ the number of permits or tickets requested by group $j$.

We add to the primitives $\bar{\mathbf{N}}$ and $\mathbf{C}$ the size vector $\mathbf{S} \in \mathbb{N}^{|\mathcal{D}|}$. We make two key assumptions:

- Size is independent from the number of feasible dates. Formally, the size of job $j$ is $S_{\rho(j)}$, where $\rho$ is a uniform random bijection from $\mathcal{D}$ to $\{1, \dots, |\mathcal{D}|\}$.
- Small amounts of overbooking are allowed. Formally, we consider it feasible to add $(j, d)$ to $\mathcal{M}$ so long as $(j, d) \in \mathcal{E}$ and $d$ has extra capacity under $\mathcal{M}$ (even if $S_{\rho(j)}$ exceeds this excess capacity).

Under these assumptions, an analogous proof holds. In particular, we reveal $\rho$ incrementally: after step $t$, we have a matching $\mathcal{M}_t$ and an injection $\rho_t : \{j : (j, d) \in \mathcal{M}_t \text{ for some } d\} \to \{1, \dots, |\mathcal{D}|\}$. In steps V1 and E1, we reveal the value $\rho_{t+1}(j_{t+1})$ (that is, the size of the job matched at time $t+1$) and ensure that $\rho_{t+1}$ is consistent with $\rho_t$ on the domain of $\rho_t$. We define

$$I_d(\mathcal{M}_t, \rho_t) = C_d - \sum_{j':(j',d)\in\mathcal{M}_t} S_{\rho_t(j')},$$

and replace (5) with

$$\mathbb{P}(B_{(t+1)j} = 1 | \mathcal{E}_t, \mathcal{M}_t, j_{t+1}, \rho_{t+1}, D_{t+1}, d_{t+1}) = \frac{N_j(\mathcal{E}_t)}{|\{d : I_d(\mathcal{M}_t) > 0\}|} \mathbf{1}(j \neq j_{t+1}) \mathbf{1}(I_{d_{t+1}}(\mathcal{M}_{t+1}) \leq 0).$$

We then couple the two chains as in Lemma 1. In order to maintain (9), we ensure that for all $t < \tau^E$ we have $d^V_{t+1} = d^E_{t+1}$ (as before) and $\rho^V_{t+1}(j^V_{t+1}) = \rho^E_{t+1}(j^E_{t+1})$. That is, the size of the group matched at time $t$ is identical in the two chains. It follows that both the number and total size of matched jobs is higher under RANDOM-VERTEX than RANDOM-EDGE.

A.2. **Non-Uniform Scheduling Constraints.** In our second extension, some days are more likely than others to be feasible for each job. For example,a programming task may be more likely to be feasible for an experienced developer than an inexperienced one. Analogously, weekends may be feasible for more hikers or theater-goers than weekdays.

We add to the primitives $\bar{\mathbf{N}}$ and $\mathbf{C}$ (and, if desired, $\mathbf{S}$) the probability vector $\mathbf{P} \in [0, 1]^{\mathcal{D}}$ with $\sum_d P_d = 1$. We make the following key assumption:

- The $\bar{\mathbf{N}}_j$ edges involving job $j$ are drawn independently (with replacement) from $\mathbf{P}$.

Formally, this requires allowing $\mathcal{E}$ to be a multiset – that is, $(j, d)$ may appear in $\mathcal{E}$ multiple times. The stages V2-V4 and E2-E4 are modified as follows.

V2/E2 Determine the multiset $\mathcal{A}_{t+1} = \{(j_{t+1}, d_{(t+1)i})\}_{i=1}^{N_j(\mathcal{M}_t)}$ by drawing the $d_{(t+1)i}$ iid, with

$$\mathbb{P}(d_{(t+1)i} = d | \mathcal{M}_t, \mathcal{E}_t, j_{t+1}) = \frac{P_d \mathbf{1}(I_d(\mathcal{M}_t) > 0)}{\sum_{d'} P_{d'} \mathbf{1}(I_{d'}(\mathcal{M}_t) > 0)}.$$

V3/E3 Choose $e_{t+1} = (j_{t+1}, d_{t+1})$ uniformly from $\mathcal{A}_{t+1}$.

V4/E4 As before, if $j = j_{t+1}$ or $I_{d_{t+1}}(\mathcal{M}_{t+1}) > 0$, then $B_{(t+1)j} = 0$. Otherwise, the value $B_{(t+1)j}$ follows a binomial distribution with parameters $N_j(\mathcal{M}_t)$ and $P_{d_{t+1}} / \sum_{d'} P_{d'} \mathbf{1}(I_{d'}(\mathcal{M}_t) > 0)$. For $j \in \mathcal{J}$, let $\mathcal{B}_{t+1}$ contain $B_{(t+1)j}$ copies of the edge $(j, d_{t+1})$.

We then couple the two chains as in Lemma 1. The only necessary modification is to the coupling of $B^V_{(t+1)j}$ and $B^E_{(t+1)j}$ following (13). In this case, draw the values $B^E_{(t+1)j}$ as described in V4/E4 and set

$$B^V_{(t+1)j} = B^E_{(t+1)\phi(j)} + \tilde{B}_{(t+1)j},$$

where $\tilde{B}_{(t+1)j} = 0$ if $j = j^V_{t+1}$ or $I_{d^V_{t+1}}(\mathcal{M}^V_{t+1}) > 0$, and otherwise $\tilde{B}_{(t+1)j}$ follows a binomial distribution with parameters $N_j(\mathcal{E}^V_t) - N_{\phi(j)}(\mathcal{E}^E_t)$ and $P_{d_{t+1}} / \sum_{d'} P_{d'} \mathbf{1}(I_{d'}(\mathcal{M}_t) > 0)$. As before, this coupling ensures that

$$N_j(\mathcal{E}^V_t \cup \mathcal{A}^V_{t+1} \cup \mathcal{B}^V_{t+1}) \geq N_{\phi(j)}(\mathcal{E}^E_t \cup \mathcal{A}^E_{t+1} \cup \mathcal{B}^E_{t+1}).$$

## Appendix B. Necessity of $C_d = 1$ in Theorem 2

The following examples illustrate that when $C_d > 1$ for some days, VI($\succ^{\mathcal{J}}, \succ^{\mathcal{D}}$) and VI($\succ^{\mathcal{J}}, \tilde{\succ}^{\mathcal{D}}$) do not necessarily lead to the same number of matches. In fact, we establish the stronger claim that the performance of RANDOM-VERTEX and RANKING differ. The following examples demonstrate that each algorithm may outperform the other.

**Example 1.** There are 2 days. For half of jobs, both days are feasible; for the other half, only one (randomly selected) day is feasible.

- If each day has capacity $\frac{1}{2}|\mathcal{J}|$, then as $|\mathcal{J}| \to \infty$, the expected fraction of unassigned jobs approaches 1/12 under RANKING, and 0 under RANDOM-VERTEX.
- If $C_{d_1} = \frac{1}{4}|\mathcal{J}|$ and $C_{d_2} = \frac{3}{4}|\mathcal{J}|$, then as $|\mathcal{J}| \to \infty$, the expected fraction of unassigned jobs approaches 1/12 under RANKING, 1/8 under RANDOM-VERTEX, and 0 under a maximum matching.

In the first case, the two days have equal capacity. Under RANDOM-VERTEX, jobs are equally likely to be scheduled on either day. By the time a day reaches its capacity, the number of remaining jobs is $o(|\mathcal{J}|)$. Under RANKING, all jobs are scheduled for the higher-priority day unless only the lower-priority day is feasible. Thus, three quarters of jobs are scheduled for the higher-priority day. After approximately $\frac{2}{3}|\mathcal{J}|$ jobs have been scheduled, this day reaches its capacity. Approximately 1/4 of the remaining jobs (1/12 of all jobs) cannot be scheduled for the low-priority day, and thus will go unscheduled.

In the second case, day $d_1$ has capacity $\frac{1}{4}|\mathcal{J}|$, which is approximately equal to the number of jobs for which only $d_1$ is feasible. By giving these jobs priority for $d_1$, a maximum matching can schedule all but $o(|\mathcal{J}|)$ jobs. Under RANDOM-VERTEX, day $d_1$ reaches its capacity after about $\frac{1}{2}|\mathcal{J}|$ jobs have been scheduled. Approximately 1/4 of the remaining jobs (1/8 of all jobs) cannot be scheduled for day $d_2$, and thus will go unscheduled. Under RANKING, if $d_1$ has higher priority than $d_2$, then it fills after approximately $\frac{1}{3}|\mathcal{J}|$ jobs have been scheduled, leaving 1/4 of the remaining jobs (1/6 of all jobs) unscheduled. However, if $d_2$ has higher priority, then neither day will fill until $o(|\mathcal{J}|)$ jobs remain. Thus, on average, RANKING leaves $\frac{1}{12}|\mathcal{J}| + o(|\mathcal{J}|)$ jobs unscheduled.

Although it is not directly related to the title of this section, we note that (i) in the first case above, RANDOM-EDGE schedules all but a vanishing fraction of jobs, and (ii) in the second, it fails to schedule $\frac{1}{6}|\mathcal{J}| + o(|\mathcal{J}|)$ jobs (day $d_1$ reaches capacity after half of jobs have been scheduled, and 1/3 of remaining jobs can only be scheduled for $d_1$).

## Appendix C. Asymptotic Performance

Theorem 1 states that on average, RANDOM-VERTEX results in a larger matching than RANDOM-EDGE. However, it is silent about the magnitude of this difference. One way to address this question is to conduct asymptotic analysis as $|\mathcal{J}|$ and $|\mathcal{D}|$ grow, with $|\mathcal{J}|/|\mathcal{D}| = \rho$ fixed. Using techniques developed in Wormald (1995, 1999), we can write down a differential equation whose solution gives the expected match size under each algorithm.

This differential equation has a clean solution for one-to-one matching in sparse Erdös-Renyi random graphs. We obtain the following expressions for the fraction of jobs that are scheduled under RANDOM-VERTEX and RANDOM-EDGE, respectively, as a function of the imbalance $\rho$ and the average number of feasible days for each job $\ell$:

$$F_V = \frac{1}{\rho}\left(1 - \frac{1}{\ell}\log\left(1 + (e^{\ell} - 1)e^{-\rho\ell}\right)\right), \tag{20}$$

$$F_E = \begin{cases} \frac{e^{(\rho-1)\ell}-1}{\rho e^{(\rho-1)\ell}-1} & : \rho > 1 \\ \frac{\ell}{1+\ell} & : \rho = 1 \\ \frac{e^{(1-\rho)\ell}-1}{e^{(1-\rho)\ell}-\rho} & : \rho < 1. \end{cases} \tag{21}$$

Theorem 3 in Mastin and Jaillet (2018) gives the expression for $F_V$ in the special case where $\rho = 1$. Their analysis directly implies (20) in the case $\rho < 1$. They do not give the expression in (20) for $\rho > 1$, although it would be possible to obtain with small modifications to their work. Dyer et al. (1993) study RANDOM-EDGE in non-bipartite Erdös-Renyi graphs, and obtain the $\ell/(\ell+1)$ expression that we claim for $\rho = 1$. We are unaware of any paper where the expression in (21) appears for general $\rho$.

For the case $\rho = 1$, these expressions imply that the fraction of unscheduled jobs is $\frac{1}{\ell+1}$ under RANDOM-EDGE and $\frac{\log(2-e^{-\ell})}{\ell} \geq \frac{\log(2)}{\ell+1}$ under RANDOM-EDGE. Both expressions are inversely proportional to $\ell$, and within a factor of $\log(2) \approx 0.69$ of each other. This suggests that the gap between these procedures is small compared to the gap between them and a maximum matching (under which the number of unscheduled jobs decreases exponentially in $\ell$). For general $\rho$, a numerical search suggests that in sparse Erdös-Renyi random graphs, the expected size of the matching produced by RANDOM-EDGE is always at least 96% of the expected size under RANDOM-VERTEX.

When we move beyond Erdös-Renyi graphs, the gap between these algorithms can be larger. For example, if there are equally many jobs and days, and half of jobs have degree 1 while the other half have degree 10, then RANDOM-EDGE expects to schedule approximately 71% of all jobs, whereas RANDOM-VERTEX schedules 78%; on this family, a maximum matching schedules approximately 89%.